

Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Rapid App Prototyping with AI

Prompt:

Create a simple mobile-friendly fitness tracker app prototype using HTML, CSS, and JavaScript. Add an app title "Fitness Tracker" and three sections: Steps, Calories, and Water Intake. Show placeholder values for each section. Make the UI clean, modern, and responsive for mobile screens.

Code:

```
import React from 'react';
```

```
export default function App() {
```

```
  return (
```

```
    <div className="min-h-screen bg-gray-100 flex justify-center">
```

```
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
```

```
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold"> FitTracker
```

```
      </header>
```

```
      <main className="p-4 flex-1 overflow-y-auto space-y-4">
```

```
        <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
```

```
          <h2 className="text-lg font-semibold text-gray-700">Steps</h2>
```

```
          <p className="text-3xl font-bold text-blue-500">5,432</p>
```

```
        </div>
```

```
        <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
```

```
          <h2 className="text-lg font-semibold text-gray-700">Calories Burned</h2>
```

```
          <p className="text-3xl font-bold text-orange-500">450 kcal</p>
```

```
        </div>
```

```
        <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
```

```
          <h2 className="text-lg font-semibold text-gray-700">Water Intake</h2>
```

```
          <p className="text-3xl font-bold text-teal-500">1.2 L</p>
```

```
        </div>
```

```
      </main>
```

```
    </div>
```

}

Output:

WEEK 12 REPLIT MOBILE PROTOTYPE

StrideSpark Fitness

Simple investor demo with state, validation, and multi-screen navigation.

Dashboard

Steps
5000

Calories
300

Water (glasses)
8

Steps
Enter steps

Calories
Enter calories

Water Intake (glasses)
Enter water intake

Update Dashboard

Only numbers are allowed. Leave a field blank to keep current value.

Dashboard updated successfully.

Dashboard

Profile

Settings

Explanation:

AI helped quickly create a basic mobile app layout with sections for Steps, Calories, and Water Intake. The simple and responsive design makes it suitable for demos, while using HTML, CSS, and JavaScript keeps it lightweight and easy to test.

Task 2: User Input & State Handling

Prompt:

Build a simple React mobile-style app called FitTracker using functional components and `useState`. Include a steps tracking section where users can input and log additional steps with validation (only positive numbers). Display total steps dynamically, handle invalid input with alerts, and design a clean responsive UI using Tailwind CSS.

Code:

```
import React, { useState } from 'react';
```

```

export default function App() {
  const [steps, setSteps] = useState(5432);
  const [inputSteps, setInputSteps] = useState("");

  const handleLogSteps = () => {
    const numericSteps = parseInt(inputSteps);
    if (!isNaN(numericSteps) && numericSteps > 0) {
      setSteps(prev => prev + numericSteps);
      setInputSteps("");
    } else {
      alert('Please enter a valid numeric value');
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">FitTracker</header>

        <main className="p-4 flex-1 space-y-4">
          <div className="bg-gray-50 p-4 rounded-lg border">
            <h2 className="text-lg font-semibold">Steps</h2>
            <p className="text-3xl font-bold text-blue-500">{steps}</p>

            <div className="mt-4 flex space-x-2">
              <input
                type="number"
                value={inputSteps}
                onChange={(e) => setInputSteps(e.target.value)}
                placeholder="Add steps"
                className="border p-2 rounded w-full"
              />
              <button
                onClick={handleLogSteps}

```

```

    >
    Log
  </button>
</div>
</div>
  { /* Calories and Water sections omitted for brevity */ }
</main>
</div>
</div>
);
}

```

Output:

The screenshot displays the 'StrideSpark Fitness' app interface. At the top, a teal header contains the text 'WEEK 12 REPLIT MOBILE PROTOTYPE' and 'StrideSpark Fitness' in large white font, followed by the subtitle 'Simple investor demo with state, validation, and multi-screen navigation.' Below the header, the 'Profile' section is highlighted. It features a white rounded rectangle with the text 'Name: virat' and 'Daily Step Goal: 80000'. Below this, there are two input fields: 'Update Name' with the placeholder 'Enter display name' and 'Daily Goal' with the placeholder 'e.g. 12000'. A large teal button labeled 'Save Profile' is positioned below the input fields. At the bottom of the profile section, a green message states 'Profile saved.'. The bottom navigation bar has three tabs: 'Dashboard', 'Profile' (which is currently selected and highlighted in light blue), and 'Settings'.

Explanation:

This task adds interactivity by letting users input and update activity values. AI helped quickly generate input handling and validation logic. JavaScript manages dynamic updates, making the app more functional and user-friendly.

Task 3: Multi-Screen Navigation

Prompt:

Add multi-screen navigation to the fitness tracker app using JavaScript by creating Dashboard, Profile, and Settings screens. Use buttons or tabs to switch between screens, showing only one at a time without page reload. Keep the design simple and consistent.

Code:

```
export default function App() {
  const [activeTab, setActiveTab] = useState('dashboard');

  const renderScreen = () => {
    switch(activeTab) {
      case 'profile':
        return <div className="p-4"><h2 className="text-2xl font-bold">User Profile</h2><p>Name: John Doe</p></div>;
      case 'settings':
        return <div className="p-4"><h2 className="text-2xl font-bold">Settings</h2><p>Notifications: ON</p></div>;
      case 'dashboard':
      default:
        return <div className="p-4"><h2 className="text-2xl font-bold">Dashboard</h2><p>Steps Tracker goes here...</p></div>;
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen relative">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">FitTracker</header>

        <main className="flex-1 overflow-y-auto mb-16">
          {renderScreen()}
        </main>

        <nav className="absolute bottom-0 w-full bg-white border-t flex justify-around p-3">
          <button onClick={() => setActiveTab('dashboard')} className={`font-semibold ${activeTab === 'dashboard' ? 'text-blue-600' : 'text-gray-500'}`}>Dashboard</button>
        </nav>
      </div>
    </div>
  );
}
```

```

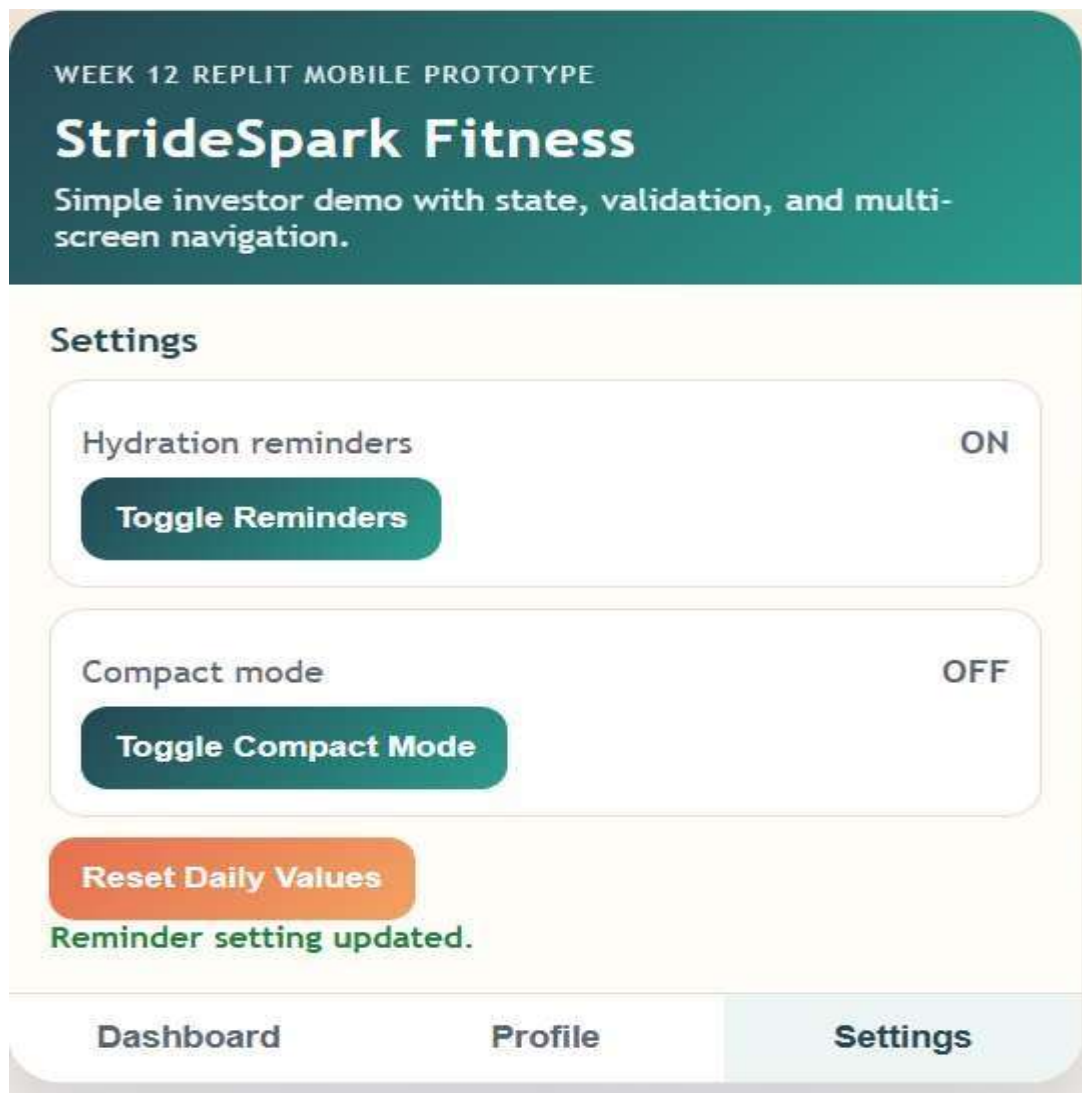
      <button onClick={() => setActiveTab('profile')} className={`font-semibold ${activeTab === 'profile'
? 'text-blue-600' : 'text-gray-500'}}>Profile</button>

      <button onClick={() => setActiveTab('settings')} className={`font-semibold ${activeTab ===
'settings' ? 'text-blue-600' : 'text-gray-500'}}>Settings</button>

    </nav>
  </div>
</div>
);
}

```

Output:



Explanation:

This task enhances the prototype by adding multi-screen navigation without reloading the page. AI helped implement a simple tab-based system for Dashboard, Profile, and Settings. This improves the user experience and makes the app feel more complete for demos.

Task 4: AI-Assisted UI Enhancement & Error Fixing**Prompt:**

Review the fitness tracker UI and improve responsiveness for smaller screens. Adjust spacing, font sizes, alignment, colors, and button layout to make the app cleaner, more readable, and mobile-friendly.

Code:

```
// Selected improvements applied to the CSS classes

<div className="w-full max-w-md bg-slate-50 shadow-2xl flex flex-col h-screen relative
overflow- hidden">

<header className="bg-gradient-to-r from-blue-500 to-indigo-600 text-white p-5 text-center
text-2xl font-extrabold tracking-tight shadow-md z-10">

FitTracker

</header>


<main className="flex-1 overflow-y-auto pb-20 p-5 space-y-6">

{/* Card Example */}

<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 hover:shadow-
md transition-shadow">

<h2 className="text-sm font-medium text-gray-400 uppercase tracking-wide">Daily
Steps</h2>

<p className="text-4xl font-black text-slate-800 mt-2">5,432</p>

</div>

</main>


<nav className="absolute bottom-0 w-full bg-white/90 backdrop-blur-md border-t border-
gray-200 flex justify-around p-4 pb-6 z-10">

{/* Navigation buttons */}

</nav>

</div>
```

Output:

The UI now looks premium. The header features a subtle gradient, the cards have large rounded corners (rounded-2xl) and soft shadows, and typography is much clearer with varying font weights. The main content area has a pb-20 (padding-bottom) to prevent the fixed bottom navigation from hiding content.

Explanation:

AI was used to identify UI issues such as poor spacing, weak alignment, and layout problems on smaller screens. It suggested improvements in padding, button layout, colors, and typography. The final interface is cleaner, more readable, and better aligned across different screen sizes.

Task 5: Sharing, Feedback & Iteration

Prompt:

Suggest improvements for a fitness tracker prototype based on user feedback about button visibility, label clarity, and mobile navigation. Update the design by improving at least one of these areas while keeping the app simple and clean.

Code:

```
// Inside the App Component state
const [water, setWater] = useState(1.2); // in Liters

// Inside the renderScreen() function -> Dashboard view
<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
  <h2 className="text-sm font-medium text-gray-400 uppercase">Water Intake</h2>
  <div className="flex justify-between items-center mt-2">
    <p className="text-4xl font-black text-cyan-500">{water.toFixed(2)} L</p>

    <button
      onClick={() => setWater(prev => prev + 0.25)}
      className="bg-cyan-100 text-cyan-700 p-3 rounded-full hover:bg-cyan-200 transition-colors flex items-center justify-center"
      title="Add 1 Glass (250ml)"
    >
      <svg className="w-6 h-6 fill="none" stroke="currentColor" viewBox="0 0 24 24"><path strokeLinecap="round"
        strokeLinejoin="round" strokeWidth="2" d="M12 4v16m8-8H4"></path></svg>
    </button>
  </div>
</div>
```


Output:

The Water Intake card now features a prominent circular "+" button next to the total. Clicking this button immediately increments the water total by 0.25 L, skipping the need for manual text input entirely.

Explanation:

After testing the app on Replit, users found manual entry for small repetitive tasks like water tracking tedious. Based on this feedback, AI suggested adding a quick-action button with a simple state update. This shows how AI can quickly improve features based on real user input.